

## BACKEND READINESS AUDIT — AFYANOVA V3

# What it takes to call this an Enterprise HIS

AfyaNova V3 is unusually well-specified for its stage — fourteen bounded domains, eight architecture decision records, and documentation that reads like it was written by the team that will maintain it for a decade. The gap isn't vision. It's that a meaningful share of the invariants the docs promise — tenant isolation, ledger immutability, clinical immutability, audit tamper-evidence — are held only by convention in application code, not by anything that can't quietly be bypassed.

Four parallel audits went through the **actual source**, not the docs, across security, clinical, financial, and interoperability surfaces, and returned **83 concrete gaps** with file-level evidence — **17 severe enough to block** calling this an enterprise HIS regardless of how many domains are already scaffolded.

**17**

CRITICAL

**25**

HIGH

**36**

MEDIUM

**5**

LOW

83 findings · 12 categories · 14 domains audited · read against docs/ and ARCHITECTURE\_RULES.md

## Where to start

The eight fixes with the most downstream leverage — everything else compounds on top of these.

- 01** Wire authorization into every controller and close the self-escalation hole in Access Control
- 02** Enforce facility-scoped authorization and stop trusting the client-sent tenant header
- 03** Make the audit "hash chain" an actual chain
- 04** Stop mutating issued invoices — build the credit/debit-note path instead

- 05 Replace hardcoded prices with a real, versioned charge master
- 06 Enforce clinical immutability at the model layer, not inside one Action
- 07 Add cross-tenant isolation tests — none exist today, for a system holding PHI
- 08 Stand up backups and a deployable build before anything else ships

Tenancy

Auth &amp; RBAC

Audit

Clinical integrity

Clinical safety

Billing

Insurance

Inventory &amp; pharmacy

Interoperability

Regulatory reporting

Testing &amp; CI

Ops &amp; deployment

Critical

High

Medium

Low

## Multi-tenancy & data isolation

3 crit 1 high 1 med

### CRITICAL

#### No facility-scoped authorization

`AuthorizationService::hasPermission()` never verifies the acting user is actually assigned to the active facility, and no

`FacilityScopeMiddleware` exists to bind or check one. A caller can pass any facility ID with no verification they belong there.

### CRITICAL

#### Tenant resolution trusts an unsigned client header

`TenantContextMiddleware.php:22-40` reads `X-Tenant-ID` straight off the request with no token or signature check, and in `local` env silently falls back to `Tenant::first()` when nothing resolves — a guessable default one config mistake away from production.

### CRITICAL

#### Patient PII tables have no tenant column at all

`patient_identifiers`, `patient_contacts`, `emergency_contacts`, and `patient_relationships` carry no `tenant_id` and their models skip the `BelongsToTenant` trait — isolation depends entirely on a `patient_id` foreign key, so any direct query against these tables bypasses tenant scoping.

**HIGH****Row-Level Security is real but mostly unwired, and never actually runs**

RLS policies exist on only 27 of 39 tenant-owned tables — Patient PII, Insurance, Inventory, Lab, MAR, and surgical tables have none — and the whole mechanism is gated on `DB::getDriverName() === 'pgsql'` while the current environment runs SQLite, so it has never actually executed.

**MEDIUM****National ID and other identifiers are stored in plaintext**

Zero `encrypted` casts exist across any model in the codebase, despite the security architecture doc mandating field-level AES-256-GCM encryption for PII like NIDA numbers.

## Authentication, RBAC & platform hardening

2 crit 3 high 3 med 1 low

**CRITICAL****No controller anywhere calls `authorize()` or `Gate::`**

Only 4 Policy classes exist across 17 domain controllers, and none of them are actually invoked. Every route is gated only by generic `auth` + `verified` middleware — any logged-in user can hit billing refunds, pharmacy dispense, or clinical sign-off endpoints regardless of role.

**CRITICAL****Any authenticated user can self-escalate to tenant admin**

`AccessControlWorkspaceController`'s `assignRole` and `updatePermissions` actions perform no permission check before letting the caller grant any role — including tenant admin — or rewrite a role's permission set.

**HIGH****The "SuperAdmin bypass" is dead code, working by accident**

`AuthorizationService.php:16` checks `$user->role === 'SuperAdmin'`, but the `users` table has no `role` column at all — the check is always false, so the intended admin-bypass rule happens to hold only because the code that would break it doesn't run.

**HIGH****MFA/TOTP is a boolean with nothing behind it**

`users.two_factor_enabled` exists, but there's no secret or recovery-code storage, no TOTP package in `composer.json`, and no enroll/verify flow — despite TOTP MFA being a named Phase 1 deliverable in the roadmap.

**HIGH****No security headers middleware**

No Content-Security-Policy, HSTS, or X-Frame-Options anywhere in the application.

**MEDIUM****Session hardening is incomplete**

`SESSION_SECURE_COOKIE` is unset and there's no enforced idle-timeout, contradicting the security doc's documented 15-minute timeout for clinical and cashier sessions.

**MEDIUM****UUIDv7 mandate isn't followed**

Around 90 models use Laravel's ULID (`HasUlid`) instead of the time-ordered UUIDv7 primary keys required by ADR-007; `ramsey/uuid` and `symfony/uid` sit installed but unused.

**MEDIUM****No break-glass emergency-access pattern**

Zero references anywhere in the codebase, despite the audit architecture doc assuming one exists for emergency chart access.

**LOW****No dedicated security-hardening packages**

`composer.json` carries no 2FA, encryption-at-rest helper, or security-scanning package beyond PHPStan/Larastan.

## Audit trail & compliance

**1 crit 2 high****CRITICAL****The audit "hash chain" isn't a chain**

`Auditable.php:53-54` computes `hash('sha256', $payload . time())` from the current record alone — it never incorporates the previous entry's hash. Tampering with or deleting historical audit rows is undetectable, despite being described as an "unbroken audit blockchain."

**HIGH****Audit records trust an unvalidated client header for facility attribution**

`Auditable.php:66` writes `X-Facility-ID` straight from the request into the audit row with no cross-check, letting a client misattribute an action to the wrong facility.

**HIGH****Audit coverage has structural holes**

Only Eloquent `created` / `updated` / `deleted` events are captured, so bulk `Model::query()->update()` or raw `DB::table()` writes are invisible. There's no login/logout or chart-view auditing, and `audit_logs`' own RLS policy restricts `SELECT` only — inserts aren't protected.

**Clinical data integrity & immutability****2 crit****2 high****2 med****1 low****CRITICAL****Signed clinical notes are immutable only by convention**

The only guard against editing a signed note is an `if ($note->is_signed)` check inside `SignClinicalNoteAction`. There's no database constraint and no model-level guard — any other code path, including the controller's own creation flow, can overwrite a signed note directly.

**CRITICAL****Vitals, diagnoses, and allergies have zero immutability enforcement**

The schema carries amendment columns (`is_amendment`, `amended_*_id`) on all three, but no Action ever sets or checks them — `RecordVitalsAction` always plain-creates, and these records can be silently overwritten via ordinary Eloquent calls.

**HIGH****A technician can silently overwrite an already-verified lab result**

Both `RecordLabResultsAction` and `VerifyLabResultsAction` plain-`update()` the `LabOrderItem` with no check for a prior `verified_by_id` — there's no amended-result trail when a verified value gets replaced.

**HIGH****The WHO Surgical Safety Time-Out is recorded, never enforced**

Neither `RecordProcedureExecutionAction` nor `BookSurgicalCaseAction` reference

`who_surgical_checklists.time_out_completed_at` — a major procedure can be documented and proceed without a verified Time-Out ever having happened.

**MEDIUM****Bed admission has a race condition**

`AdmitPatientAction.php:44` checks bed availability before opening its `DB::transaction` and without a row lock — two concurrent admissions can both pass the check and double-occupy the same bed.

**MEDIUM****Deceased-patient status is never checked**

`patients.status` supports "Deceased," but no registration, encounter-start, booking, or admission Action gates against it.

**LOW****Surgical sign-out counts default to true**

`CompleteWhoChecklistAction.php:26` defaults instrument/swab counts to correct with no itemized count field behind them.

## Clinical safety, decision support & missing modules

1 crit 3 high 9 med

**CRITICAL****No radiology, imaging, DICOM, or PACS module exists**

Zero matches anywhere in the codebase, despite DICOM/PACS integration being named explicitly as an objective in the project's own vision document.

**HIGH****Allergy checking is a bypassable fuzzy string match**

`PrescribeMedicationAction.php:64-82` does a `LIKE` `'%generic_name%'` match against free-text allergen strings and **returns with no check at all** whenever `drug_class` is null. There is no drug-drug interaction checking anywhere.

**HIGH****Patient merge doesn't relink the patient's actual medical record**

`MergePatientsAction.php` reassigns only identifiers and contacts — encounters, diagnoses, allergies, invoices, and lab orders stay pointed at

...mentary, aggregated, and/or merged, and has been lost, possibly in the now-merged-away patient record, fragmenting their clinical history.

**HIGH****No referral, consent, immunization, or maternal-health workflows**

No referral management, consent/advance-directive tracking, immunization records, growth charts, or ANC/partograph support exist — several of which are Tanzanian MTUHA reporting expectations, not optional extras.

**MEDIUM****No panic/critical alerting on vital signs**

Only Lab has a critical-value flag. `clinical_vitals` has no `is_critical` column, and `RecordVitalsAction` only checks basic physiologic plausibility (e.g. temperature range), not clinically dangerous thresholds.

**MEDIUM****No persistent problem list**

`diagnoses` requires an `encounter_id` on every row, so there's no cross-encounter "active problems" view of a patient.

**MEDIUM****No medication reconciliation across encounters or admissions****MEDIUM****No clinical decision support beyond the bypassable allergy check**

No order sets, no care pathways, no structured alerts beyond vitals range validation.

**MEDIUM****No specimen or stratified reference-range models**

Reference ranges and panic thresholds live in an unstructured JSON blob on `lab_tests` with no age/sex stratification, and there's no dedicated specimen-tracking entity.

**MEDIUM****Surgical records have no implant or blood-product fields**

No implant-serial or blood/blood-product administration columns exist despite being named explicitly in the vision document's OT description.

**MEDIUM****No consumable catalog (bill-of-materials) for procedures**

Consumables are logged ad hoc per execution with no catalog-driven default list, risking missed billing items.

**MEDIUM****No patient-facing surface at all**

188 lines of `routes/web.php` contain zero patient-facing or API routes — no self-booking, no results viewing.

**MEDIUM****No nursing care plans or flowsheets beyond the MAR**

## Financial ledger & billing integrity

**2 crit 2 high 5 med****CRITICAL****No charge master or tariff engine exists**

Prices are hardcoded inline — a flat 20,000 TZS default in `GenerateInvoiceAction`, a flat 500 TZS "prototype" unit price in `PrescribeMedicationAction` — instead of coming from a versioned, effective-dated price catalog.

**CRITICAL****Issued invoices get mutated in the normal code path, not just an edge case**

`GenerateInvoiceAction` uses `firstOrCreate` against an already-"Issued" invoice, then plain-`update()`s its total whenever a second charge lands on the same encounter — directly breaking the project's own append-only invoice rule. No `CreditNote` / `DebitNote` model exists to do this correctly.

**HIGH****Ledger balance isn't actually enforced by the database**

ADR-004 claims triggers block `UPDATE` / `DELETE` on ledger tables; no such trigger or CHECK constraint exists. The in-code balance check only re-sums entries that were mechanically constructed to already match, so it could never catch a real imbalance.

**HIGH****M-Pesa is a label, not an integration**

Payment actions only map an "M-Pesa" string to a ledger account code. There's no Daraja STK-push client, webhook controller, or signature verification anywhere in the repository.



MEDIUM

**No tax/VAT calculation anywhere in Billing**

MEDIUM

**Discount authorization has a permission with nothing behind it**

`InvoicePolicy::approveDiscount()` exists, but no discount field or workflow exists for it to actually govern.

MEDIUM

**No multi-currency support**

No `currency` column on invoices or ledger tables — TZS is implicit everywhere, despite the vision document naming TZS, KES, and USD.

MEDIUM

**Refunds have no over-refund guard and misroute funds**

`IssueRefundAction` lets `paid_amount` go negative, and hardcodes the refund destination to the cash ledger account regardless of the original payment channel — misposting refunds of mobile-money or card payments.

MEDIUM

**No AR aging, GL export, or chart-of-accounts export**

Nothing here can hand financial data to an external accounting system.

## Insurance claims lifecycle

1 crit 1 high 2 med 1 low

CRITICAL

**Eligibility verification is a local stub, not the adapter pattern the architecture calls for**

`VerifyPolicyEligibilityAction` checks a local expiry date and trusts whatever biometric flag the caller sends. No NHIF or private-insurer adapter, and no external call, exists anywhere — despite this being the entire premise of the project's own ADR on Tanzanian insurance adapters.

HIGH

**Claim adjudication never touches the general ledger**

Approved claims always get the full claimed amount — no line-level partial adjudication — and adjudication never posts to

`LedgerTransaction` / `LedgerEntry`, so insurance receivables are never actually recognized in the books.

**MEDIUM****Claim generation fabricates data when real data is missing**

`GenerateClaimFromEncounterAction.php:71-83` injects a hardcoded 15,000 TZS consultation line item when no invoice actually exists.

**MEDIUM****The claims scrubber is weak**

It checks diagnosis presence, policy expiry, and provider record, but not pre-authorization enforcement, annual benefit limits, or physician license validation — all specified in the insurance architecture doc.

**LOW****Claim "submission" is just a status flip**

No actual file or API transmission artifact is produced.

## Inventory & pharmacy supply chain

1 crit

2 high

4 med

1 low

**CRITICAL****Billed and dispensed quantities are two unconnected numbers**

Prescriptions are invoiced in full at prescribe-time, and `DispenseMedicationAction` never links back to correct the invoice for partial dispense, substitution, or stockout.

**HIGH****Goods receipts and purchase orders never post to the ledger**

Inventory value increases with no corresponding payable recorded anywhere, and PO tax is hardcoded to zero.

**HIGH****The controlled-substance register isn't a real dual-signature control**

Witness ID is optional and unvalidated; the running-balance lookup has no row lock, so it races under concurrent administration; and shortfalls are silently clamped to zero instead of raising an exception.

**MEDIUM****Stock balance drift is absorbed, not reconciled**

`DispenseStockAction` silently takes `min(balance, requested)` instead of erroring when a batch and its parent balance disagree, so balances are never recomputed from the movement ledger they're supposed to derive from.

**MEDIUM****No over-receipt validation on stock transfers**

Received quantity isn't capped to dispatched quantity, allowing phantom stock to be created.

**MEDIUM****Predictive reordering fabricates a supplier as a side effect**

The reorder-point algorithm itself is real, but silently creates a fake supplier record when none exists rather than flagging the gap.

**MEDIUM****Prescription verification is a stub**

It accepts a rejection reason parameter but never persists it or the verifying pharmacist's identity.

**LOW****Purchase-order approval is single-step**

No threshold-based multi-level approval or budget/cost-center control.

## Interoperability & external integrations

1 crit 1 high 3 med

**CRITICAL****No FHIR code exists anywhere**

The FHIR facade architecture described in the docs — Patient/Observation/Condition mappers — has never been started. There isn't even an Integration domain folder.

**HIGH****Mobile money is name-only**

See Financial Ledger, above — no Daraja client, webhook, or signature verification exists for M-Pesa, Airtel Money, or Tigo Pesa.

**MEDIUM****No HL7/ASTM lab-analyzer interfacing**

All lab results are entered manually — there's no ingestion endpoint or edge-agent hook for connecting an actual analyzer.

**MEDIUM****No SMS gateway integration**

Appointment reminders and lab-ready notifications described in the integration doc are entirely unbuilt.

**MEDIUM****No real-time or broadcasting layer**

No `config/broadcasting.php`, no Reverb or Pusher — the "live queue" and "visual bed map" have no transport for real-time updates.

## Regulatory reporting — MTUHA & DHIS2

**3 high 2 med****HIGH****The regulatory reporting schema was never migrated**

None of `report_definitions`, `report_snapshots`, or `dhis2_data_elements` exist. "MTUHA" appears in the codebase only as an inventory stationery line item, not as a digital register.

**HIGH****What exists instead isn't the actual regulatory deliverable**

The analytics Actions compute a generic "top-10 diagnosis" list, not the MoH Kitabu cha 1/2/5 register formats regulators actually require.

**HIGH****No DHIS2 export or API client exists anywhere****MEDIUM****Analytics won't hold up across a multi-facility network**

Existing report Actions load entire unfiltered result sets into PHP memory on every request, with no caching or pre-aggregated snapshotting.

**MEDIUM****No PDF or spreadsheet export capability**

No `dompdf`, `snappy`, or `maatwebsite/excel` in the dependency tree — nothing here can be handed to a regulator or printed as an invoice or prescription.

## Testing coverage & CI quality gates

**1 crit 2 high 2 med**

**CRITICAL****Zero cross-tenant isolation or authorization-boundary tests exist**

No test anywhere asserts that a user from Tenant 1 can't read Tenant 2's data — despite this being explicitly mandated in the project's own testing strategy for a multi-tenant system holding PHI.

**HIGH****There is no invariant test suite at all**

The mandated ledger-balance, zero-negative-stock, and clinical-immutability regression tests don't exist as a directory or as tests — which is exactly why the ledger-mutation and immutability findings above shipped uncaught.

**HIGH****CI silently swallows front-end type errors**

`npm run type-check || true` in the workflow means TypeScript/Vue type failures can never fail a build.

**MEDIUM****The Audit domain has no dedicated feature test****MEDIUM****Mutation testing is an orphaned dependency, not a working gate**

`pestphp/pest-plugin-mutate` sits in the lockfile but isn't declared in `composer.json`, has no config, and is invoked by no script or CI step.

## Observability, backup, deployment & scaling

2 crit 3 high 3 med 1 low

**CRITICAL****No backup automation of any kind**

No `spatie/laravel-backup` or equivalent, no disaster-recovery or point-in-time-recovery script anywhere in the repository — for a system whose entire premise is being trusted with hospital data.